

Propagation Delay Between Two Ground Stations Communicating via GEO Satellite

Dominic Alessi

August 2026

1 Introduction

Geostationary Earth Orbit (GEO) satellites must orbit very high above the Earth's surface to remain stationary over a point along the equator. This means that the propagation delay, the time it takes for the radio signal to physically travel from ground station 1 to ground station 2, can exceed one quarter of a second, which is particularly noticeable for real-time audio channels. This report presents the calculations for computing that propagation delay.

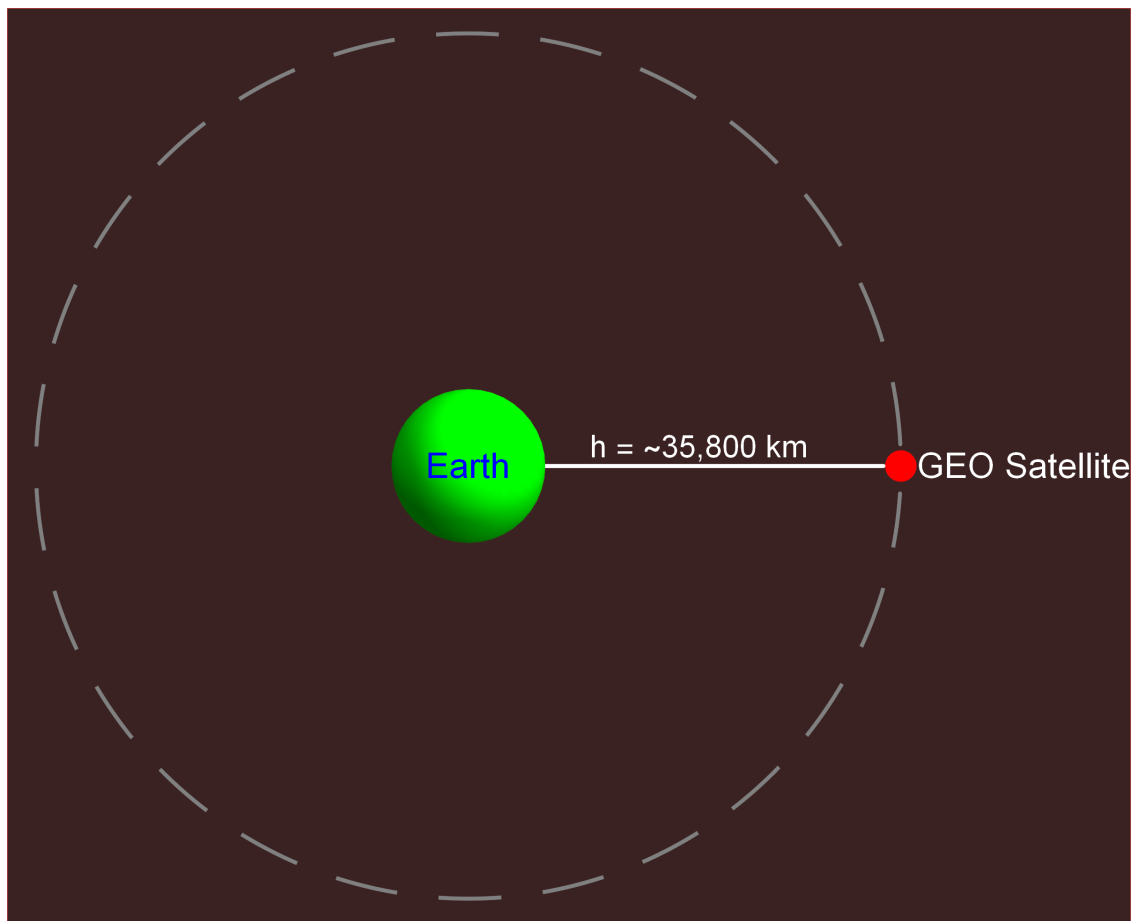


Figure 1: A to-scale depiction of a GEO around Earth. The height required for GEO is very great, thereby contributing to the relatively long propagation delay compared to LEO satellite communication systems. This image was created with Mathematica.

Given the locations of both ground stations and the GEO satellite, the MATLAB program `GEO_Propagation_Delay_Calculator.m` computes the propagation delay. The program (1) takes user inputs in latitude and longitude and converts them to Cartesian, (2) calculates the angle between satellite and ground station location vectors (3) confirms that both ground stations are within line-of-sight (LOS) of the satellite, and (4) calculates the distances between both ground stations and the satellite, and then the propagation delay.

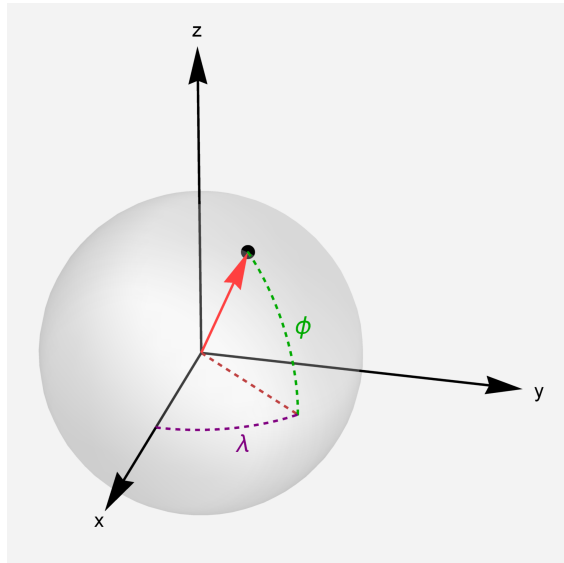
2 Assumptions

Before diving into calculations, these two assumptions were made: (1) the radio wave's propagation velocity is the speed of light and does not change as it travels through Earth's atmosphere, and (2) the radio wave travels in a perfectly straight line from transmitter to receiver. These assumptions are defensible for these calculations because the amount in which they affect the final calculated propagation delay is negligible simply due to the distance from GEO. Lastly, any time required for signal processing or packet switching is neglected. This is only a calculation of the propagation delay.

3 Calculations

3.1 Conversion to Cartesian

To convert latitude (ϕ) and longitude (λ) to a Cartesian coordinate system, the axes must be defined. It was selected to make the Z-axis align with Earth's axis of rotation and to have the X-axis pass through both the center of Earth and Null Island (0°N, 0°E). Naturally, to satisfy $\vec{z} = \vec{x} \times \vec{y}$ this means that the Y-axis passes through both Earth's center and (0°N, 90°E). The conversion equations are as follows:



$$x = R \cos(\lambda) \cos(\phi) \quad (1)$$

$$y = R \sin(\lambda) \cos(\phi) \quad (2)$$

$$z = R \sin(\phi) \quad (3)$$

Here ϕ and λ respectively represent latitude and longitude, and R is the object's distance from the Earth's center. For ground stations R is Earth's radius, and for the satellite it is the sum of its orbital height and Earth's radius, $R + h$. Also note that this coordinate system is distinct from a spherical coordinates since Φ measures from the xy-plane rather than the z-axis. Although, the equations for converting to Cartesian are quite similar.

Figure 2: A visualization of latitude and longitude (created with Mathematica).

3.2 Angle of Separation, β

Now with the locations of ground station 1, ground station 2, and the satellite in vector form as $\vec{g}_1$, $\vec{g}_2$, and $\vec{s}$, the angle between any ground station-satellite pair, β_i , can be computed with the dot product identity:

$$\beta_i = \cos^{-1}\left(\frac{\vec{g}_i \cdot \vec{s}}{|\vec{g}_i||\vec{s}|}\right) \quad (4)$$

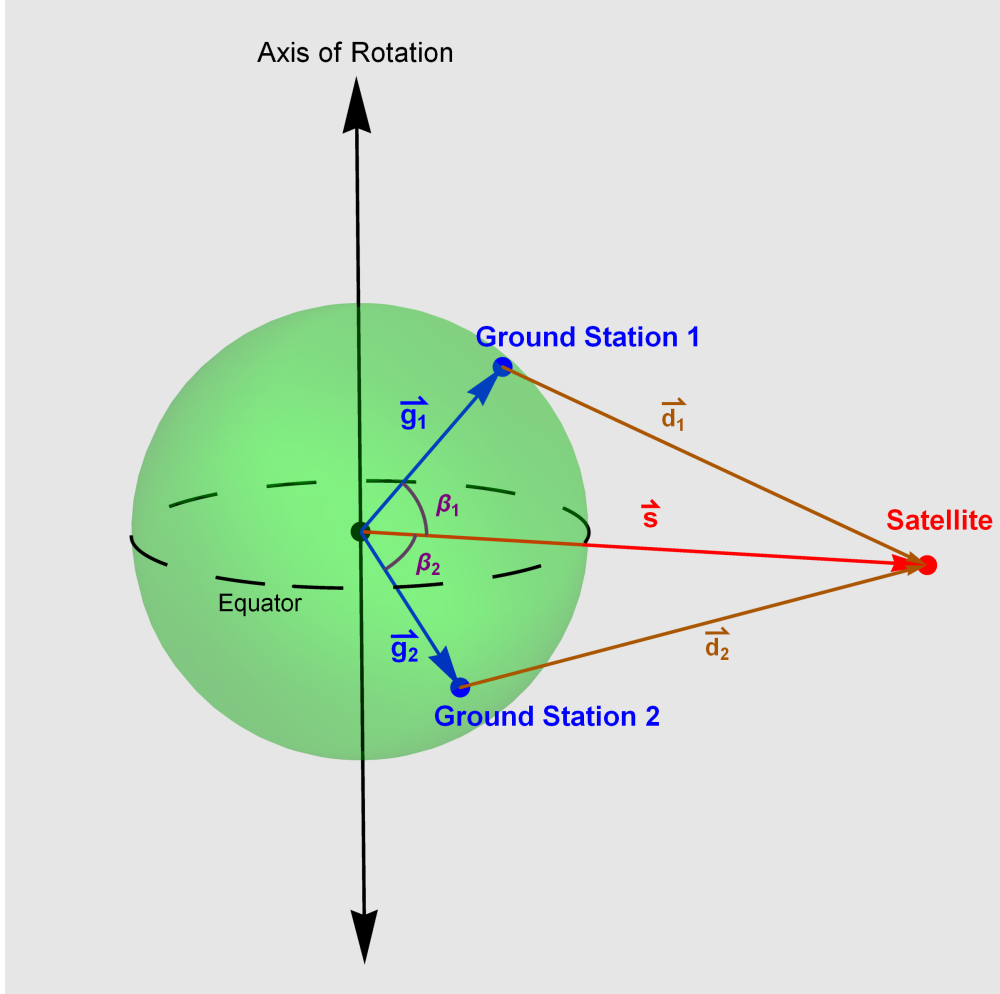


Figure 3: A visual depiction of the ground station-satellite system. β_i is the angle of separation between its ground station and the satellite, $\vec{d}_i$ is the displacement vector from its ground station to the satellite.

Next, to assert that both ground stations are within LOS of the satellite, we check that both β_1 and β_2 are less than β_{max} , the angle of separation in the extreme case where the satellite is exactly on the visual horizon of the ground station. In this case a right-triangle with vertices at the center of Earth, the ground station, and the satellite is formed. This results in equation 5 giving β_{max} .

$$\beta_{max} = \cos^{-1}\left(\frac{R_e}{R_e + h}\right) \quad (5)$$

Earth's radius is known to be 6,371,000 meters, and the height required for GEO can be quickly derived from Newton's second law:

3.2.1 Newtonian Physics

$$F_c = m_s a_c, \quad a_c = \frac{V_T^2}{R_{orbit}} \quad (6)$$

$$G\left(\frac{m_s m_e}{(h + R_e)^2}\right) = m_s \left(\frac{V_T^2}{h + R_e}\right), \quad V_T = (R_e + h)\omega_s \quad (7)$$

A key component of GEO is that Earth's angular velocity equals the satellite's, i.e. $\boxed{\omega_e = \omega_s}$. Thus, by algebraic manipulation:

$$h = \sqrt[3]{\frac{G \cdot m_e}{\omega_e^2}} - R_e \quad (8)$$

The mass of the earth is 5.9722×10^{24} kilograms, the angular velocity of the Earth is 7.29×10^{-5} radians per second, and the universal gravitational constant is $6.6743 \times 10^{-11} \left[\frac{m^3}{kg \cdot s^2}\right]$. Plugging these values into equation 8 means that the height required for GEO orbit is 35,801,401.91 meters, or approximately 35.8 thousand kilometers.

3.3 Asserting Line-of-Sight Condition

Now with the value of h in hand, it can easily be determined that the maximum angle of separation for LOS propagation between the ground station and the satellite to work is:

$$\beta_{max} = \cos^{-1}\left(\frac{6.371 \times 10^6}{(6.371 \times 10^6) + (35,801,401.91)}\right) = 81.311^\circ \quad (9)$$

Thus, the angle of separation between either ground station and the satellite must be less than 81.311° to be within LOS of each other.

3.4 Distance and Propagation Delay

From here, having location vectors $\vec{g}_1$, $\vec{g}_2$, and $\vec{s}$, and having asserted that both ground stations are within LOS of the satellite, all that remains to calculate the propagation delay is determining the distance the signal must travel and then dividing by the speed of light, 299,792,458 meters per second. Both distances d_1 and d_2 are easily found by vector subtraction:

$$\vec{d}_i = \vec{s} - \vec{g}_i \quad (10)$$

Now dividing by c for the duration of the propagation delay T_p :

$$T_p = \frac{|\vec{d}_1| + |\vec{d}_2|}{c} \quad (11)$$

4 Conclusion

A key tradeoff of GEO satellites compared to other satellite communications systems is the higher propagation delay in exchange for covering a larger area. In fact, their area of coverage is so great that it would only require three GEO satellites to cover nearly the entire globe. Figure 4 is a map of the areas covered by a GEO satellite located 100°W, table 1 provides the propagation delays between various ground stations for that satellite, and figure 5 is a plot of the propagation delay between a ground station and the GEO satellite as a function of their angle of separation, β .

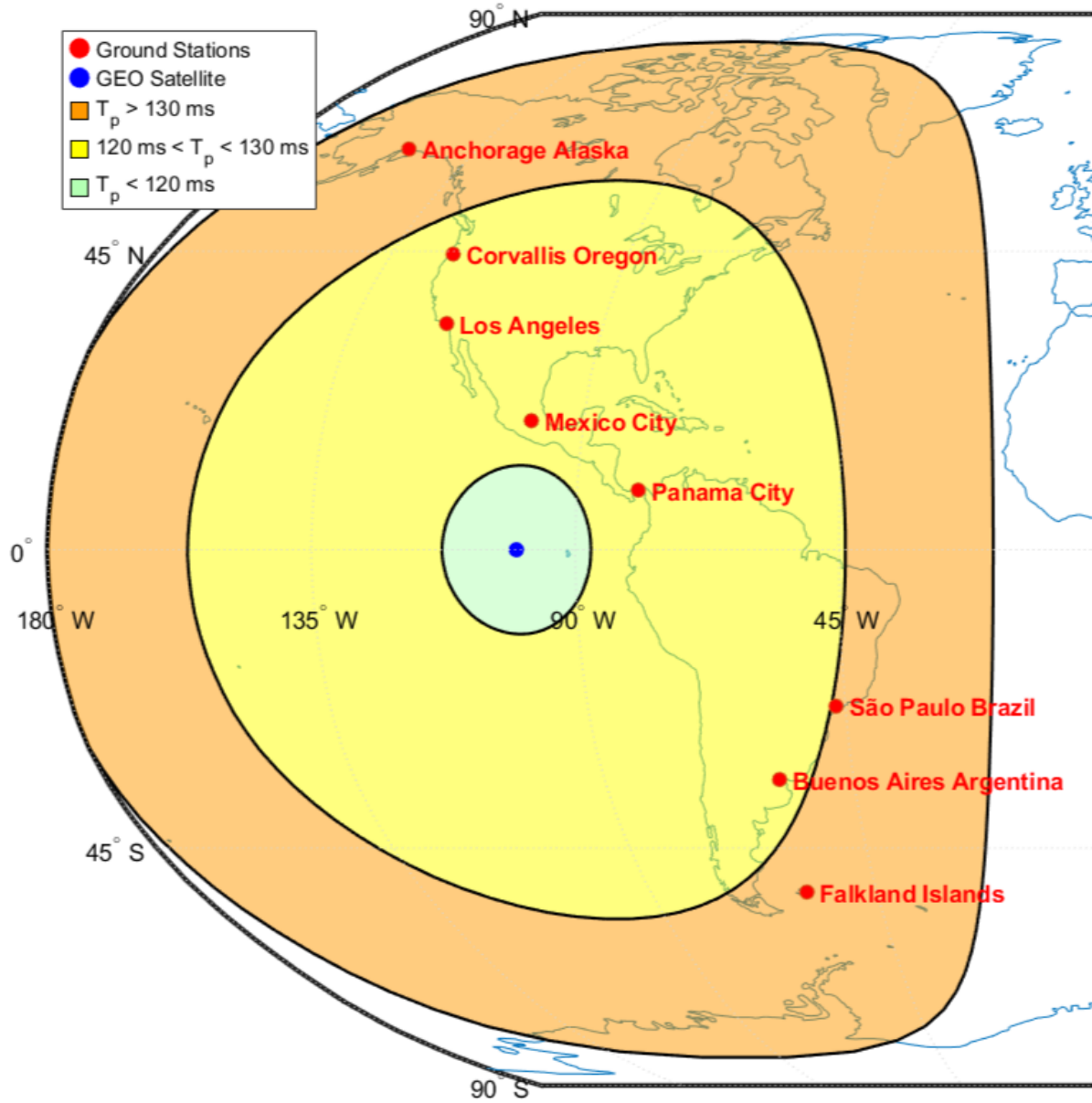


Figure 4: A map of the area of service provided by a GEO satellite located 100°W. A single GEO satellite may provide service to the whole of North and South America.

Table 1: Real World Propagation Delay Examples

Ground Station 1	Ground Station 2	Propagation Delay
Anchorage Alaska (61.2°N,149.9°W)	Falkland Islands (51.81°S,58.94°W)	267.65 ms
Corvallis Oregon (44.57°N,123.29°W)	Buenos Aires Argentina (34.6°S,58.38°W)	256.42 ms
Los Angeles (34.05°N,118.25°W)	São Paulo Brazil (23.56°S,46.64°W)	254.85 ms
Mexico City (19.43°N,99.12°W)	Panama City (8.98°N,79.52°W)	242.06 ms

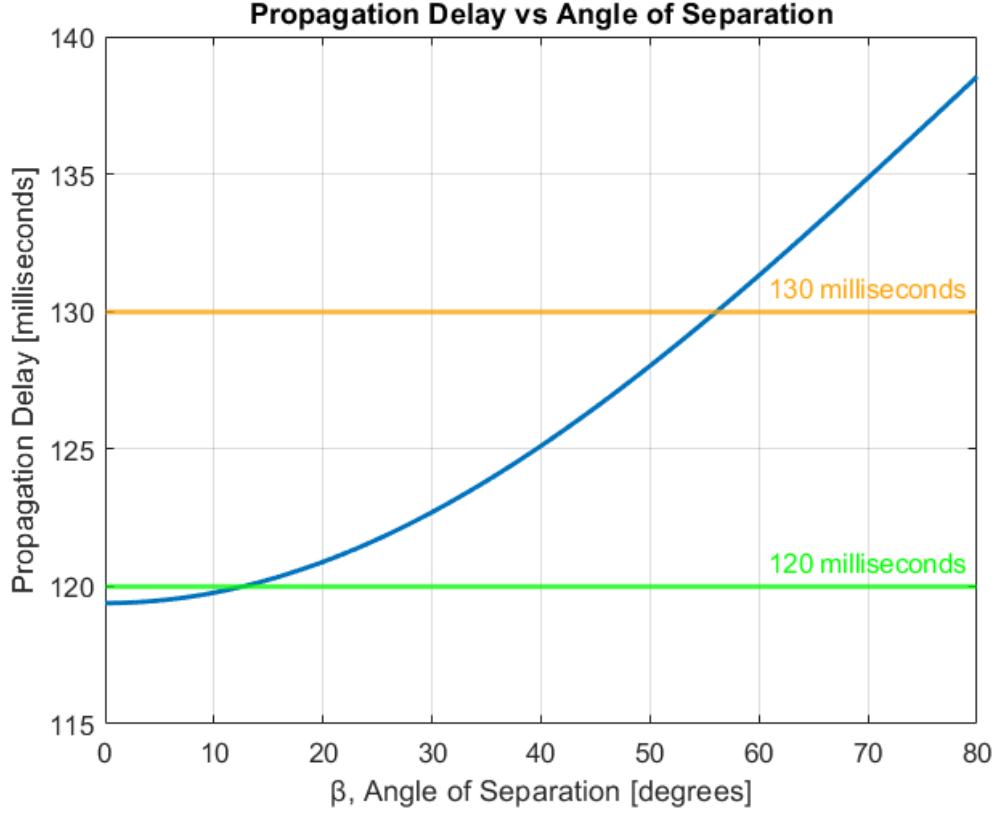


Figure 5: A plot of the propagation delay as a function of the ground station's angle of separation β from the satellite.

5 Description of Each MATLAB File

- `GEO_Propagation_Delay_Calculator.m`: this is the function that actually computes the propagation delay using the locations of both ground stations and the satellite as parameters.
- `GEO_Propagation_Delay_Demo.m`: prompts the user for ground station latitudes and longitudes, and the satellite's longitude, then calls `GEO_Propagation_Delay_Calculator()` to compute the propagation delay. It also makes a useful map to help visualize the range of a GEO satellite.
- `propagationDelayPlot.m`: generates figure 5.
- `realWorldExamples.m`: generates figure 4 and the information in table 1.
- `parseData.m`: helper function for reading `places.txt` which has information about cities and their latitudes and longitudes and returns a 2D matrix.
- `blankToBlank.m`: a helper function for printing propagation delay information.
- `eastOrWest.m`: a helper function for adding "°E" to the end of positive longitudes, and removing the negative sign and adding "°W" to negative longitudes.
- `northOrSouth.m`: a helper function for adding "°N" to the end of positive latitudes, and removing the negative sign and adding "°S" to negative latitudes.